

1) What is the output of the following Python code?

```
def add_item(item, target=[]):  
    target.append(item)  
    return target
```

```
print(add_item(1))  
print(add_item(2))
```

- A) [1]
[2]
- B) [1]
[1, 2]
- C) [1]
[2, 1]
- D) [1, 2]
[1, 2]

2) What will be the output of executing this snippet?

```
funcs = [lambda x: x * i for i in range(3)]  
print([f(2) for f in funcs])
```

- A) [0, 2, 4]
- B) [4, 4, 4]
- C) [0, 1, 2]
- D) [2, 2, 2]

3) What is the expected output of the slice operation below?

```
nums = [10, 20, 30, 40, 50, 60]  
print(nums[5:1:-2])
```

- A) [60, 40]
- B) [60, 40, 20]
- C) [50, 30]
- D) [60, 50, 40]

4) What does the following comparison code output in standard CPython?

```
a = 256  
b = 256
```

```
c = 257
d = 257
print(a is b, c is d)
```

5) What happens when executing the code below?

```
d = {}
t = (1, 2, [3, 4])
d[t] = 'value'
print(d)
```

- A) {(1, 2, [3, 4]): 'value'}
- B) TypeError: unhashable type: 'list'
- C) ValueError: mutable element in tuple
- D) KeyError: invalid tuple key

6) Predict the output of the nested function execution:

```
x = 10
def outer():
    x = 20
    def inner():
        nonlocal x
        x += 5
        print(x, end=' ')
    inner()
    print(x, end=' ')
```

```
outer()
print(x)
```

- A) 25 20 10
- B) 25 25 10
- C) 25 25 25
- D) 15 20 10

7) What will be printed when consuming the generator below?

```
gen = (x * x for x in range(3))
print(next(gen), next(gen))
print(list(gen))
```

- A) 0 1
[0, 1, 4]
- B) 0 1
[4]
- C) 0 1
[]
- D) 0 1
[1, 4]

8) What is the output according to Method Resolution Order (MRO)?

```
class A:  
    def show(self):  
        print('A', end="")
```

```
class B(A):  
    def show(self):  
        print('B', end="")  
        super().show()
```

```
class C(A):  
    def show(self):  
        print('C', end="")  
        super().show()
```

```
class D(B, C):  
    def show(self):  
        print('D', end="")  
        super().show()
```

D().show()

- A) DBCA
- B) DBAC
- C) DCBA
- D) DBA

9) What will the following code print?

```
def func(a, b, c, d):  
    print(a + b + c + d)
```

```
args = [1, 2]  
kwargs = {'d': 4, 'c': 3}  
func(*args, **kwargs)
```

- A) 10
- B) TypeError: func() got multiple values for argument
- C) SyntaxError: positional argument follows keyword argument
- D) [1, 2, 3, 4]

10) What is printed when calling the decorated function below?

```
def my_decorator(func):  
    def wrapper():  
        print('1', end="")  
        func()  
        print('2', end="")  
    return wrapper
```

```
@my_decorator  
def say_hello():  
    print('3', end="")
```

```
say_hello()
```

- A) 312
- B) 132
- C) 123
- D) 321

11) What is the return value of the function call check()?

```
def check():  
    try:  
        return 1  
    finally:  
        return 2
```

```
print(check())
```

- A) 1
- B) 2
- C) 1 2
- D) UnboundLocalError

12) What will be printed by the snippet below?

```
class Test:
    count = 0
    def __init__(self):
        Test.count += 1

t1 = Test()
t2 = Test()
t1.count = 10
print(t1.count, t2.count, Test.count)
```

- A) 10 2 2
- B) 10 10 10
- C) 2 2 2
- D) 10 2 10

13) What is the value of 'a' after executing the following copy operations?

```
import copy
a = [[1, 2], [3, 4]]
b = copy.copy(a)
b[0][0] = 99
b.append([5, 6])
print(a)
```

- A) [[1, 2], [3, 4]]
- B) [[99, 2], [3, 4]]
- C) [[99, 2], [3, 4], [5, 6]]
- D) [[1, 2], [3, 4], [5, 6]]

14) What are the output values of any(a) and all(b)?

```
a = [0, [], "", None, False]
b = [1, [0], 'False', True]
print(any(a), all(b))
```

- A) False True
- B) False False
- C) True True
- D) True False

15) What will be the output of this slice operation?

```
s = 'Python'  
print(s[1:5:-1])
```

- A) 'ytho'
- B) 'ohty'
- C) " (Empty string)
- D) 'nohtyP'

16) Predict the output of s.x and s.y:

```
class Sample:  
    def __getattr__(self, name):  
        return f'Missing: {name}'
```

```
s = Sample()  
s.x = 100  
print(s.x, s.y)
```

- A) 100 Missing: y
- B) Missing: x Missing: y
- C) 100 None
- D) AttributeError: 'Sample' object has no attribute 'y'

17) What does zip(*pairs) produce in the code snippet?

```
pairs = [(1, 'a'), (2, 'b'), (3, 'c')]  
nums, chars = zip(*pairs)  
print(nums)
```

- A) [(1, 2, 3)]
- B) (1, 2, 3)
- C) [1, 2, 3]
- D) ((1, 'a'), (2, 'b'), (3, 'c'))

18) What are the boolean values evaluated in the print statement?

```
class Parent: pass
class Child(Parent): pass
```

```
c = Child()
print(isinstance(c, Parent), type(c) == Parent)
```

- A) True True
- B) True False
- C) False True

19) What is the final value of d['a'] and len(d)?

```
keys = ['a', 'b', 'a', 'c']
values = [1, 2, 3, 4]
d = {k: v for k, v in zip(keys, values)}
print(d['a'], len(d))
```

- A) 1 4
- B) 3 3
- C) 1 3
- D) 3 4

20) What is the evaluation result of the chained comparison?

```
print(1 < 2 < 3 == 3 > 2)
```

- A) True
- B) False
- C) TypeError: chained comparison mismatch
- D) SyntaxError: invalid comparison chain

21)

What will be the output of the following code snippet?

```
print(type(5 / 2))
print(type(5 // 2))
```

- A) Float and int
- B) Int and float
- C) Float and float

D) Int ant int

22) What will be the output of the following code snippet?

```
a = 3
b = 1
print(a, b)
a, b = b, a
print(a, b)
```

- A) 3 1 , 1 1
- B) 3 1 , 1 3
- C) 1 3 , 1 3
- D) 1 3 , 3 1

23) s = {1, 2, 3, 3, 2, 4, 5, 5}

```
print(s)
```

24) word = "Python Programming"

```
n = len(word)
word1 = word.upper()
word2 = word.lower()
converted_word = ""
for i in range(n):
    if i % 2 == 0:
        converted_word += word2[i]
    else:
        converted_word += word1[i]
print(converted_word)
```

25) a = [[], "abc", [0], 1, 0]

```
print(list(filter(bool, a)))
```

26) def solve(a, b):

```
    return b if a == 0 else solve(b % a, a)
```

```
print(solve(20, 50))
```

27) def func():

```
    global value
```

```
    value = "Local"
```

```
value = "Global"
```

```
func()
```



```

print(value)
28) def solve(a):
    a=[1,3,5]
a=[2,4,6]
print(a)
solve(a)
print(a)
29) my_list = ['Multiple of 6' if i % 6 == 0
               else 'Multiple of 2' if i % 2 == 0
               else 'Multiple of 3' if i % 3 == 0
               else i for i in range(1, 20)]
print(my_list)
30) def outer_function(): # Fixed space here
    value = "Say cheese to"

    def inner_function1():
        nonlocal value
        value = "Python"

    def inner_function2():
        global value
        value = "Engineers"

    inner_function1()
    inner_function2()
    return value

value = "Programming"
result = outer_function()
print(value, result)

```

ANSWERS:

- 1) B) [1] and [1, 2]

Explanation: Default argument values are evaluated once when the function is defined, not each time it is called. The same list instance 'target' is mutated across successive calls.

- 2) B) [4, 4, 4]

Explanation: Python closures bind variables by reference, not value (late binding). When the functions are called, the loop has completed and 'i' has a final value of 2. Thus $2 * 2 = 4$ for all functions.

- 3) A) [60, 40]

Explanation: The slice starts at index 5 (60) and moves backward with a step of -2 until index 1 (exclusive). Index 5 gives 60, index 3 gives 40. Index 1 is excluded.

- 4) B) True False

Explanation: CPython pre-allocates and caches integers in the range [-5, 256]. Hence 'a is b' evaluates to True because they reference the same object. For 257, separate objects are created, making 'c is d' False.

- 5) B) TypeError: unhashable type: 'list'

Explanation: Dictionary keys must be hashable. Although tuples are immutable, this tuple contains a mutable list [3, 4], which makes the entire tuple unhashable.

- 6) B) 25 25 10

Explanation: 'nonlocal x' causes inner() to modify the variable 'x' in the enclosing outer() scope ($20 + 5 = 25$). The global variable 'x' remains untouched at 10.

- 7) B) 0 1 and [4]

Explanation: The first two calls to next(gen) consume values 0 ($0*0$) and 1 ($1*1$). Calling list(gen) consumes the remaining item 4 ($2*2$), leaving [4].

- 8) A) DBCA

Explanation: The MRO for class D is $D \rightarrow B \rightarrow C \rightarrow A$. Following super() calls through this linearization order produces 'D', then 'B', then 'C', then 'A'.

- 9) A) 10

Explanation: *args unpacks [1, 2] as positional parameters 'a' and 'b'. **kwargs unpacks {'d': 4, 'c': 3} as keyword parameters. Total sum: $1 + 2 + 3 + 4 = 10$.

10)B) 132

Explanation: @my_decorator wraps say_hello in wrapper(). Calling say_hello() executes wrapper(): prints '1', then calls original func() which prints '3', then prints '2'.

11)B) 2

Explanation: The 'finally' block is guaranteed to execute before leaving the function. If a return statement is present in 'finally', it overrides any return statement in 'try' or 'except'.

12) A) 10 2 2

Explanation: Instantiation updates Test.count to 2. 't1.count = 10' creates an instance variable on t1 shadowing the class variable. t2.count and Test.count still refer to the class variable (2).

13) B) [[99, 2], [3, 4]]

Explanation: copy.copy() creates a shallow copy. Modifying nested mutable objects (b[0][0]=99) affects 'a' because nested lists are shared. Appending a new list to 'b' does not affect 'a'.

14) A) False True

Explanation: In 'a', all elements evaluate to False, so any(a) is False. In 'b', 1 is truthy, non-empty list [0] is truthy, non-empty string 'False' is truthy, and True is truthy, so all(b) is True.

15)C) " (Empty string)

Explanation: With a negative step (-1), the slice moves backward from start index 1 towards stop index 5. Since index 1 (y) is less than 5 (n), moving backward never reaches 5, resulting in an empty string.

16)A) 100 Missing: y

Explanation: __getattr__ is only invoked when an attribute is NOT found in the instance dict. 's.x' exists (100). 's.y' is missing, triggering __getattr__('y').

17)B) (1, 2, 3)

Explanation: zip(*pairs) unpacks the 3 tuples as positional arguments to zip(), separating elements into two tuples: nums = (1, 2, 3) and chars = ('a', 'b', 'c').

18)B) True False

Explanation: isinstance(c, Parent) returns True because Child inherits from Parent. type(c) == Parent returns False because type(c) is explicitly Child, not Parent.

19)B) 3 3

Explanation: zip produces ('a',1), ('b',2), ('a',3), ('c',4). When key 'a' is encountered a second time, value 3 overwrites 1. The resulting dict has 3 keys ('a', 'b', 'c').

20)A) True

Explanation: Python chains comparisons as implicit 'and' conditions: (1 < 2) and (2 < 3) and (3 == 3) and (3 > 2). Since every sub-expression is True, the whole expression evaluates to True.

21)A) float and int

The 1st expression performs standard division so the result is stored as a float type. The 2nd expression performs integer division so the result is stored as int type.

22)B) 31 , 13

23){1,2,3,4,5}

24) pYtHoN PrOgRaMmInG

25) ['abc', [0], 1]

The above code filters all the elements from list a, which evaluates to boolean value true, i.e. any non empty string, list or non-zero value is accepted.

26) 10, GCD of two numbers recursively

27) Local as it is declared as global

28)[2,4,6], [2,4,6]

29)[1, 'Multiple of 2', 'Multiple of 3', 'Multiple of 2', 5, 'Multiple of 6', 7, 'Multiple of 2', 'Multiple of 3', 'Multiple of 2', 11, 'Multiple of 6', 13, 'Multiple of 2', 'Multiple of 3', 'Multiple of 2', 17, 'Multiple of 6', 19]

30)Engineers Python

nonlocal refers to variable in enclosing function (outer_function) so "Say cheese to python", value is declared global in inner_function2() so value is changed to "Engineers"